

Code Complexity Analysis

Us014 – Ford-Fulkerson

(Maximum Flow)

Index

- Function 1: FordFulkerson (constructor)
 - Function 2: initializeResidualGraph
 - Function 3: computeMaxFlow
 - Function 4: dfs
 - Function 5: getMaxFlow
-

1. Pseudocode

Function 1: FordFulkerson (constructor)

```
IF graph == null OR source == null OR sink == null
    THROW IllegalArgumentException

Initialize source, sink
maxFlow = 0
Initialize residualGraph

CALL initializeResidualGraph(graph)
CALL computeMaxFlow
```

Function 2: initializeResidualGraph

```
FOR each edge (u → v) in graph
    capacity = edge.capacity OR 0
    residualGraph[u][v] = capacity
    residualGraph[v][u] = 0    // backward edge
```

Function 3: computeMaxFlow

```
Initialize parent map

WHILE dfs(parent) == true
    pathFlow =  $\infty$ 
    v = sink

    WHILE v != source
        u = parent[v]
        pathFlow = min(pathFlow, residualGraph[u][v])
        v = u

    IF pathFlow == 0
        BREAK

    maxFlow += pathFlow

    v = sink
    WHILE v != source
        u = parent[v]
        residualGraph[u][v] -= pathFlow
        residualGraph[v][u] += pathFlow
        v = u
```

Function 4: dfs

```

Clear parent
Initialize visited set
Initialize stack

Push source into stack
visited.add(source)
parent[source] = null

WHILE stack not empty
    u = stack.pop()

    IF u == sink
        RETURN true

    FOR each neighbor v of u in residualGraph
        IF v not visited AND residualCapacity(u,v) > 0
            parent[v] = u
            visited.add(v)
            stack.push(v)

RETURN false

```

Function 5: getMaxFlow

```

RETURN maxFlow

```

2. Time Complexity

FordFulkerson (constructor)

Operation	Description	Avg Case	Notes
Input validation	Null checks	1	Constant
initializeResidualGraph	Build residual graph	E	Linear over edges
computeMaxFlow	Main algorithm loop	$E \cdot f^*$	Dominant
TOTAL		$O(E \cdot f^*)$	

initializeResidualGraph

Operation	Description	Avg Case	Notes
Iterate over edges	Read original graph edges	E	One pass

Operation	Description	Avg Case	Notes
Insert forward edge	Map put	1	HashMap
Insert backward edge	Map putIfAbsent	1	HashMap
TOTAL		$O(E)$	

computeMaxFlow

Operation	Description	Worst Case	Notes
dfs	Find augmenting path	$V + E$	Graph traversal
Path flow calculation	Backtrack from sink to source	V	Path length
Residual graph update	Update forward + backward edges	V	Path length
Number of iterations	Flow increments	f^*	
TOTAL		$O(E \cdot f^*)$	

dfs

Operation	Description	Worst Case
Clear parent map	Reset path	V
Stack operations	Push / pop vertices	V
Visited set operations	HashSet insert / lookup	V
Iterate neighbors	Traverse residual edges	E
TOTAL		$O(V + E)$

getMaxFlow

Operation	Description	Avg
Return	Getter method	1
TOTAL		$O(1)$

3. Best, Average and Worst Case (Big-O)

Case	Complexity	Explanation
Best Case	$O(E)$	No path exists between source and sink or only one DFS is executed
Average Case	$O(k \cdot E)$	k = number of augmenting paths, usually small in real networks
Worst Case	$O(E \cdot f^*)$	Each DFS increases flow by 1 unit (pseudo-polynomial)

4. Determinism Analysis

Aspect	Deterministic?	Explanation
Final max flow value	☐ Yes	Guaranteed by Max-Flow Min-Cut Theorem

Aspect	Deterministic?	Explanation
Execution path	☐ No	DFS explores neighbors stored in HashMap (iteration order not guaranteed)
Number of iterations	☐ No	Depends on chosen augmenting paths
Execution time	☐ No	Different paths → different iterations

5. Space Complexity

Structure	Complexity	Description
Residual graph	$O(V + E)$	Stores forward and backward edges
Parent map	$O(V)$	Path reconstruction
Visited set	$O(V)$	DFS bookkeeping
Stack	$O(V)$	Iterative DFS
TOTAL	$O(V + E)$	

6. Final Summary

- Deterministic Result: **Yes**
 - Deterministic Execution: **No**
 - Time Complexity (Worst Case): **$O(E \cdot f^*)$**
 - Space Complexity: **$O(V + E)$**
-

Legend:

V = number of vertices (stations)

E = number of edges (railway lines)

f^* = maximum flow value

Time Complexity - Risk-Aware Shortest Paths

1. Time Complexity of `getBellmanFordPath`

Let:

- V = number of vertices

- E = number of edges

The method follows the standard Bellman-Ford structure, with extra work for negative-cycle detection and reconstruction.

1.1 Initialization

Iterates over all vertices to set initial distances.

- $O(V)$

1.2 Relaxation Phase

Outer loop runs $V - 1$ times.

Inner loop scans all E edges each time.

- Dominant term: $O(V \times E)$

1.3 Negative-Cycle Detection

A single pass over all edges.

- $O(E)$

1.4 Negative-Cycle Reconstruction (if triggered)

- Backtracking through predecessors: $O(V)$
- Building the cycle: $O(V)$
- For each cycle edge, searching through all edges: $O(V \times E)$
- Worst-case: $O(V \times E)$

1.5 Normal Path Reconstruction

Backtracks from destination to origin, at most V vertices.

- $O(V)$

Overall Complexity of `getBellmanFordPath`

- $O(V \times E)$ (dominant term)
-

2. Time Complexity of `printBellmanFordPath`

This method only iterates over the precomputed results.

2.1 With a Negative Cycle

Iterates through cycle vertices/edges, at most **V**.

- $O(V)$

2.2 With a Standard Path

Iterates through the path, also at most **V**.

- $O(V)$

Final Summary

Method	Time Complexity
<code>getBellmanFordPath</code>	$O(V \times E)$
<code>printBellmanFordPath</code>	$O(V)$